ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic - Machine Learning from Disaster (Kaggle), train.csv

Model: Random Forest inside a scikit-learn Pipeline (imputation + scaling + one-hot encoding), saved as model.pkl with Joblib.

Split: 712 training / 179 unseen test records. **Test accuracy:** 81.01% (train accuracy 86.24%).

Task 6: Observations

1. How does Online Inference differ from Batch Inference?

Online inference predicts for a single record (or a very small number) immediately, as soon as the request arrives, so it is optimised for low latency - e.g. a user checking one passenger's survival chance through an API. Batch inference collects many records (here 20) and scores them together in one call, usually offline or on a schedule, and stores the results (batch_predictions.csv). It is optimised for throughput and efficiency rather than instant response.

2. What is the difference between Streaming Inference and Batch Inference?

Batch inference works on a fixed, already-collected dataset that is processed all at once. Streaming inference handles data that arrives continuously: each record is scored individually as it arrives (here one record every 1 second) and the result is available immediately, without waiting for the full dataset. Streaming gives near real-time results on unbounded data, while batch gives results only after the whole group is processed.

3. Why is FastAPI used for model serving?

FastAPI is fast (built on Starlette/ASGI), simple to write, and uses Python type hints. Pydantic gives automatic request validation, so invalid input is rejected with a clear 422 error before it reaches the model. It auto-generates interactive Swagger UI (/docs) documentation for testing, supports async, and makes the model reachable by any client (web, mobile, other services) through a standard REST interface.

4. Why should the same trained model be reused for all inference strategies?

Reusing one saved pipeline guarantees consistency: the same preprocessing and the same learned weights are applied whether a record is scored online, in a batch, in a stream or via the API, so the same input always gives the same prediction. It avoids training-serving skew, saves the cost of retraining, makes results reproducible and comparable, and means that only one artifact (model.pkl) has to be versioned, tested and maintained.